

PONTIFICIA UNIVERSIDAD JAVERIANA

FACULTAD DE INGENIERÍA
INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

LABORATORIO 1

Implementación de Monolito con patrón MVC, DAO y API
REST en .NET 7

Santiago Mesa
Camilo Peñuela
Sergio Orosman Parra

Docente: Andrés Sánchez

Asignatura: Arquitectura de Software

11 de mayo de 2026
Bogotá D.C., Colombia

Índice

1	Marco Conceptual	2
1.1	Tecnologías Utilizadas	2
1.2	Arquitectura Monolítica	2
1.3	Patrón MVC (Model-View-Controller)	2
1.4	Patrón DAO / Repository	3
2	Diseño del Sistema	3
2.1	Modelo Relacional de la Base de Datos	3
2.2	Arquitectura y Despliegue con Docker	4
2.3	Flujo de la Aplicación (MVC)	6
3	Procedimiento	6
3.1	Limpieza del Script de la Base de Datos	6
3.2	Corrección de las Rutas (Routing)	6
3.3	Creación de las Vistas Faltantes	7
3.4	Instrucciones de Despliegue Paso a Paso	7
4	Conclusiones	7
5	Referencias	7

1. Marco Conceptual

En este primer laboratorio se aplicaron varios conceptos fundamentales de arquitectura de software para construir una aplicación web completa y funcional.

1.1. Tecnologías Utilizadas

Para desarrollar el proyecto se utilizaron las siguientes herramientas:

- **.NET 7 (ASP.NET Core):** El framework principal donde se programó toda la lógica en C#.
- **Entity Framework Core:** Permitió conectar la base de datos con el código sin tener que escribir consultas SQL manualmente, ya que mapea las tablas directamente a clases de C#.
- **SQL Server (Azure SQL Edge):** La base de datos relacional elegida. Se utilizó la imagen de Azure SQL Edge para garantizar su correcto funcionamiento en computadores con procesadores ARM (como los Mac).
- **Docker y Docker Compose:** Herramientas clave que permitieron *empaquetar* todo el proyecto (base de datos y aplicación) en contenedores para que el código funcione igual en cualquier computador.
- **Swagger:** Una librería que generó automáticamente una interfaz visual para probar y documentar la API REST de forma interactiva.

1.2. Arquitectura Monolítica

Se implementó una arquitectura monolítica, lo que significa que tanto la interfaz visual (las páginas web) como la lógica de negocio y la conexión a la base de datos están alojadas dentro del mismo proyecto y se ejecutan juntas como una sola unidad.

1.3. Patrón MVC (Model-View-Controller)

El código se organizó utilizando el patrón MVC, dividiéndolo en tres partes fundamentales:

- **Modelos:** Clases en C# que representan la información (como Persona o Profesión).
- **Vistas:** Archivos HTML/C# (.cshtml) que conforman las pantallas que visualiza el usuario.
- **Controladores:** Los encargados de recibir las peticiones del usuario, buscar la información necesaria y enviarla a la vista.

1.4. Patrón DAO / Repository

Para mantener el código ordenado, se utilizó el patrón Repository. Esto significa que se crearon clases específicas (por ejemplo, `PersonaRepository`) dedicadas únicamente a comunicarse con la base de datos. De esta forma, los controladores solo solicitan los datos al repositorio sin preocuparse de cómo se consiguen a nivel interno.

API REST en el Monolito: Además de las páginas web tradicionales (MVC), el monolito desarrollado también cuenta con controladores que funcionan como una API REST. Estos endpoints devuelven información pura en formato JSON, lo cual resulta muy útil si más adelante se requiere conectar una aplicación móvil o un frontend moderno como React o Angular.

2. Diseño del Sistema

En esta sección se detalla cómo se diseñó el sistema, la estructura de la base de datos y cómo fluye la información.

2.1. Modelo Relacional de la Base de Datos

La base de datos almacena información de personas, sus profesiones y sus teléfonos. El esquema relacional implementado es el siguiente:

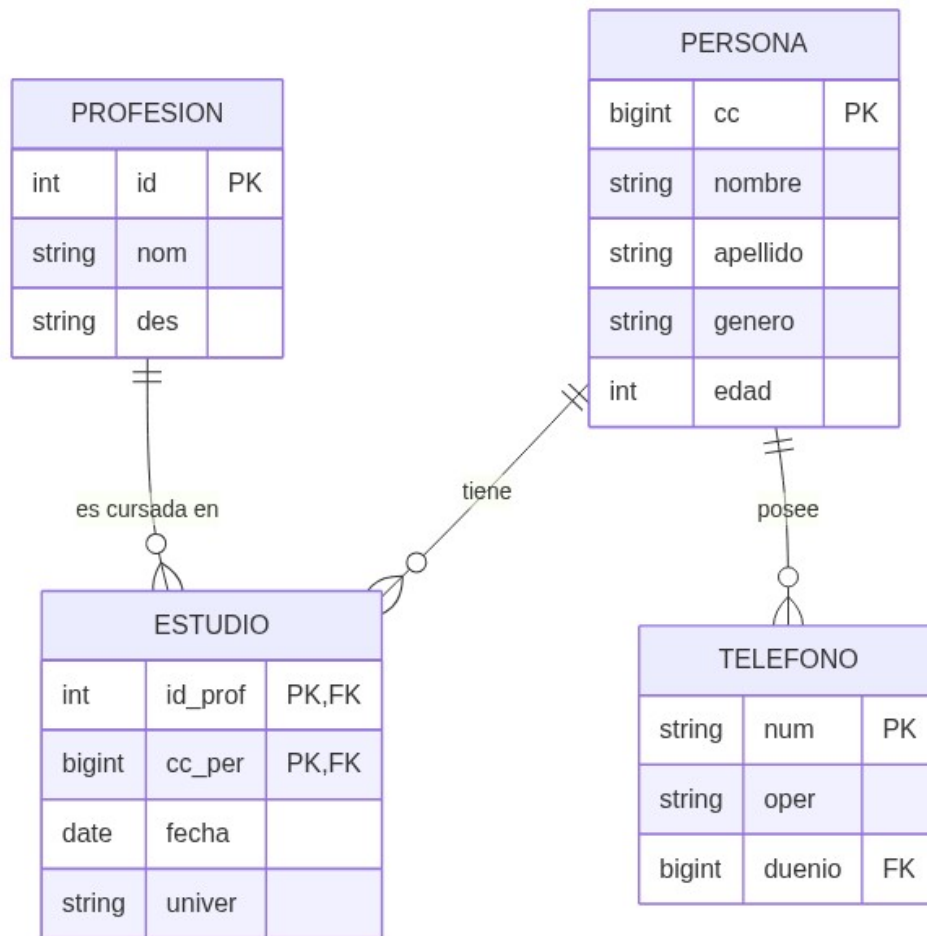


Figura 1: Diagrama Entidad-Relación de la base de datos

Como se observa en la Figura 1, la tabla **PERSONA** actúa como la entidad principal. Para relacionar a una persona con una profesión se creó la tabla intermedia **ESTUDIO**, debido a que una persona puede tener varias profesiones y una misma profesión puede ser estudiada por muchas personas. Adicionalmente, la tabla **TELEFONO** permite guardar múltiples números de contacto asociados a una misma persona.

2.2. Arquitectura y Despliegue con Docker

Uno de los mayores retos del laboratorio consistía en lograr que el proyecto se ejecutara sin fallos en cualquier computador, evitando los típicos problemas de conectividad en el puerto **localhost** entre sistemas Windows y macOS. Para solucionarlo, se recurrió a Docker Compose.

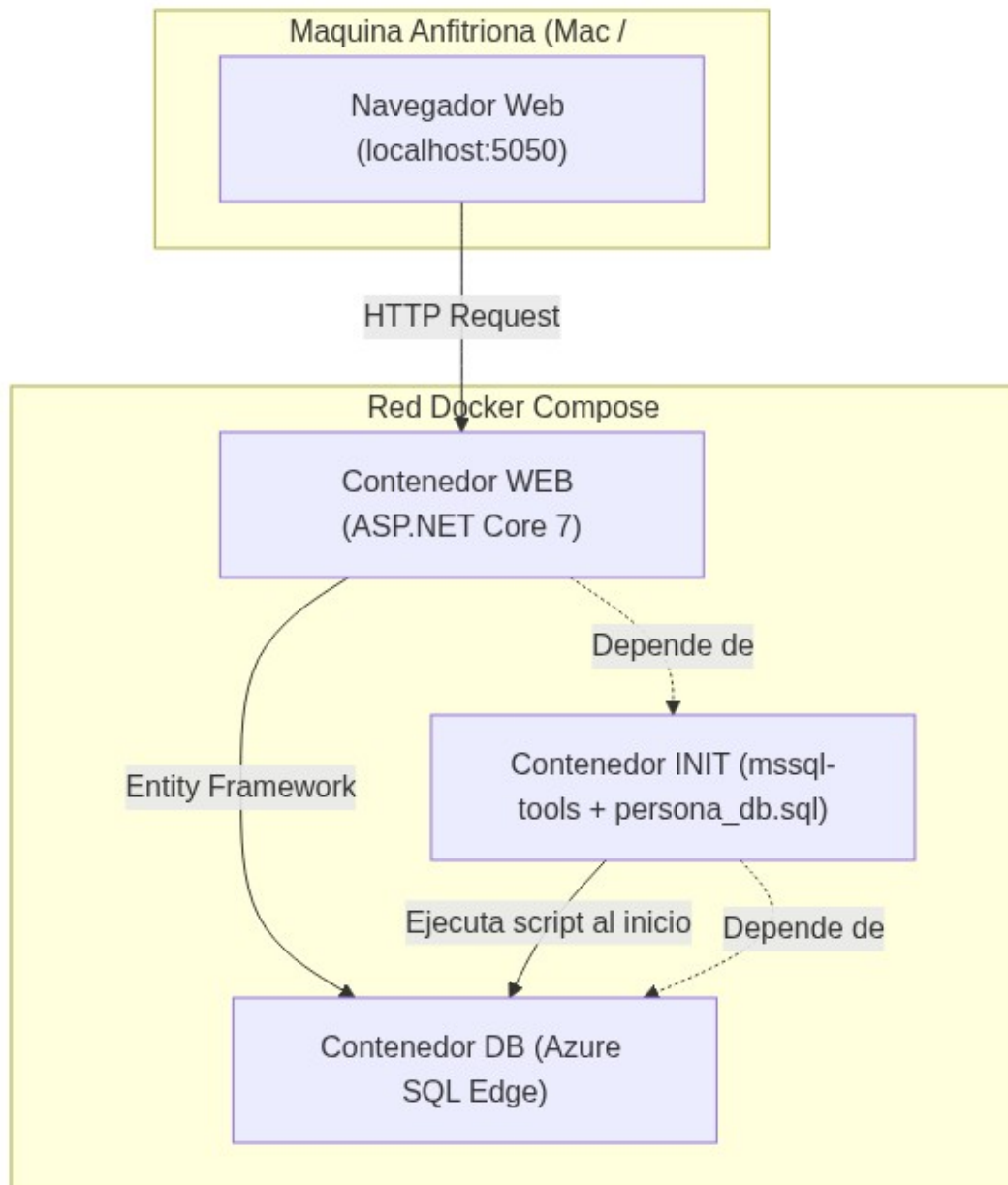


Figura 2: Arquitectura de contenedores en Docker

En la Figura 2 se expone cómo se configuraron tres contenedores interconectados:

1. **Contenedor DB:** Es el servidor de SQL Server que almacena los datos reales.
2. **Contenedor INIT:** Es un contenedor temporal. Su único propósito es esperar a que la base de datos inicie correctamente, ejecutar el script `persona_db.sql` para crear las tablas y llenarlas con los datos de prueba, y finalmente apagarse de forma automática.
3. **Contenedor WEB:** Contiene la aplicación construida en ASP.NET. Este contenedor se conecta a la base de datos mediante la red interna de Docker, y permite al usuario acceder a la plataforma a través de la dirección `localhost:5050` en el navegador.

2.3. Flujo de la Aplicación (MVC)

Para comprender el comportamiento del sistema al navegar por la aplicación, se elaboró el siguiente diagrama de secuencia.

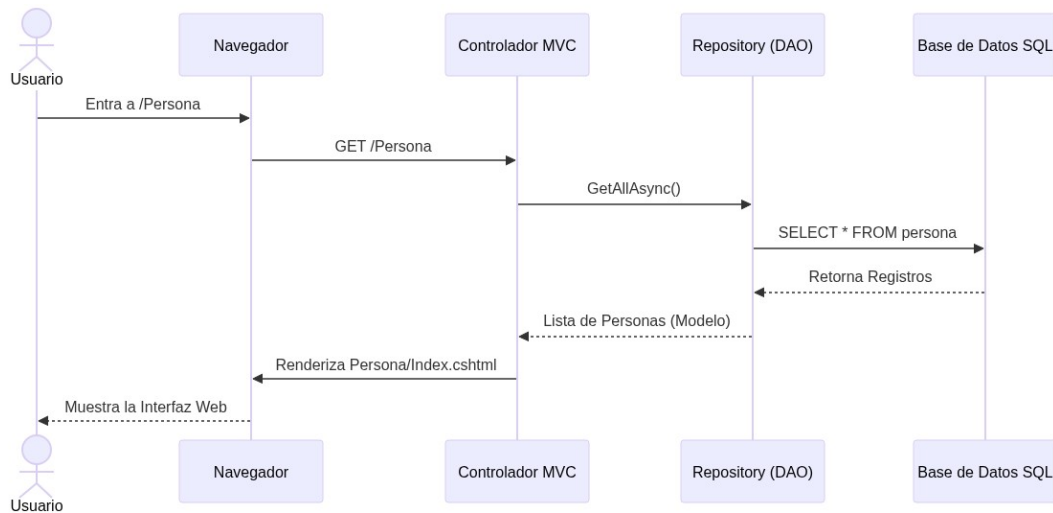


Figura 3: Diagrama de Secuencia de una petición web

En la Figura 3 se detalla el flujo de un caso de uso común: 1. El usuario accede a la sección de Personas (`/Persona`) desde el navegador. 2. El sistema de enrutamiento de .NET dirige esa petición al controlador `PersonaController`. 3. El controlador se comunica con el Repositorio y solicita la lista completa de personas. 4. El Repositorio envía la consulta respectiva a la base de datos SQL y recibe los registros. 5. Luego, el controlador toma esos datos y los transfiere a la Vista correspondiente (`Index.cshtml`). 6. Finalmente, la Vista genera el código HTML estructurado (la tabla con los datos) y se lo muestra al usuario.

3. Procedimiento

Durante el desarrollo del laboratorio se presentaron algunos inconvenientes técnicos que fue necesario solucionar para alcanzar el funcionamiento total de la aplicación:

3.1. Limpieza del Script de la Base de Datos

El archivo SQL suministrado inicialmente con el proyecto (`persona_db.sql`) presentaba errores de sintaxis y conflictos con los saltos de línea (CRLF) propios de Windows, lo que provocaba que Docker fallara al intentar crear las tablas en macOS. Se reescribió el script completo, limpiando los caracteres problemáticos y añadiendo comandos separadores como `GO` para asegurar que la inyección de los datos de prueba se ejecutara automáticamente, sin requerir intervención manual.

3.2. Corrección de las Rutas (Routing)

En un inicio, la aplicación solo podía resolver las URLs correspondientes a la API REST, debido a que el archivo `Program.cs` estaba configurado únicamente con la instrucción `app.MapControllers()`. Se agregó la configuración `MapControllerRoute` para indicarle a .NET la forma correcta de encontrar e interpretar las páginas web del esquema MVC:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

3.3. Creación de las Vistas Faltantes

Se desarrollaron las 14 vistas (.cshtml) requeridas para completar el CRUD de las entidades *Estudio*, *Profesión* y *Teléfono*. Estas interfaces se maquetaron empleando el framework Bootstrap 5 y se ubicaron en sus carpetas correspondientes dentro del directorio **Views**.

3.4. Instrucciones de Despliegue Paso a Paso

Para ejecutar el proyecto desde cero en cualquier computador, se deben seguir estos pasos desde la terminal (ubicándose en la carpeta **Laboratorio1** del repositorio):

1. Asegurarse de tener instalado y corriendo **Docker Desktop**.
2. Ejecutar el comando `docker-compose up -d --build`. Este comando descarga las imágenes, compila el código .NET, ejecuta el script SQL de inicialización y levanta los servicios.
3. Esperar un par de minutos mientras el contenedor `db_init` configura la base de datos automáticamente.
4. Abrir un navegador e ingresar a `http://localhost:5050` para usar la plataforma web (MVC), o a `http://localhost:5050/swagger` para probar la API REST.
5. Para detener el proyecto, basta con ejecutar `docker-compose down`.

4. Conclusiones

- **Ventajas de usar Docker:** Se comprobó que empaquetar el ecosistema con Docker Compose ahorra una gran cantidad de problemas de compatibilidad. Al tener SQL Server y .NET aislados en contenedores, se asegura que el proyecto se ejecute de manera idéntica en cualquier equipo que clone el repositorio.
- **Monolito versátil:** El desarrollo de este taller demostró que dentro de un mismo proyecto en .NET es completamente factible mantener funcionando una aplicación web completa basada en vistas (MVC), en conjunto con una API REST ([ApiController]) lista para proveer información a otros sistemas externos.
- **Orden en el código:** La implementación del patrón Repository resultó ser una decisión fundamental. Gracias a esta arquitectura, si en algún momento se decidiera migrar de SQL Server a otra base de datos diferente, bastaría con modificar únicamente los repositorios, sin alterar una sola línea de código en los controladores.

5. Referencias

- Microsoft Documentation. (2023). *Tutorial: Create a web API with ASP.NET Core*.
- Guía del Laboratorio 1 proporcionada por el docente de Arquitectura de Software.
- Fowler, M. (2015). *Microservices and Monoliths*.